ItsDangerous — Developer Guide

ItsDangerous is a small, dependency-free Python library from the Pallets project (the Flask team). It **signs** data so you can hand it to an untrusted party — a browser cookie, a URL, an email link — and later verify that it came back unmodified.

> **Signed is not encrypted.** Anyone holding a token can base64-decode it and read the payload. ItsDangerous only proves the data wasn't *changed*. Never put secrets in a token.

Package	`itsdangerous` (version in tree: `2.3.0.dev`, unreleased)
Python	3.10+ (3.8/3.9 dropped in 2.3.0)
Runtime dependencies	None (stdlib only: `hmac`, `hashlib`, `json`, `zlib`, `base64`)
License	BSD-3-Clause
Upstream docs	https://itsdangerous.palletsprojects.com/

---

1. Quick start

```
from itsdangerous import URLSafeSerializer

auth_s = URLSafeSerializer("secret key", salt="auth")
token = auth_s.dumps({"id": 5, "name": "itsdangerous"})
# 'eyJpZCI6NSwibmFtZSI6Iml0c2Rhbmdlcm91cyJ9.6YP6T0BaO67XP--9UzTrmurXSmg'

auth_s.loads(token)          # {'id': 5, 'name': 'itsdangerous'}
auth_s.loads(token + "x")    # raises itsdangerous.BadSignature
```

With an expiry (the most common real-world case — password reset links, email confirmation):

```
from itsdangerous import URLSafeTimedSerializer, SignatureExpired, BadSignature

s = URLSafeTimedSerializer(SECRET_KEY, salt="password-reset")
token = s.dumps({"user_id": 42})

try:
    data = s.loads(token, max_age=3600)      # seconds
except SignatureExpired:
    ...  # valid signature, but too old
except BadSignature:
    ...  # tampered, wrong key, or wrong salt
```

---

2. Core concepts

Signer vs. Serializer

There are two layers:

1. **Signer** — signs and verifies raw `bytes`/`str`. Output: `value` + `sep` + `signature`.
2. **Serializer** — wraps a Signer. Serializes any object (JSON by default) to bytes, then signs it. API mirrors `json`: `dumps` / `loads`.

You almost always want a Serializer. Each layer has a timestamped variant:

Need	Class
Sign bytes	`Signer`
Sign bytes + record signing time	`TimestampSigner`

Need	Class
Sign objects (JSON)	`Serializer`
Sign objects + expiry	`TimedSerializer`
Sign objects, URL/cookie-safe, compressed	`URLSafeSerializer`
...plus expiry	`URLSafeTimedSerializer`

Secret key

- Secures every signature. Anyone who knows it can forge tokens.
- Use long random bytes, load from the environment, never commit it. Generate one: `python3 -c 'import os; print(os.urandom(16).hex())'`
- Changing it invalidates all existing tokens.
- **Key rotation:** pass a list, oldest → newest. The **last** key signs; verification tries every key from newest to oldest.

```
s = URLSafeSerializer(["old-key", "current-key"])
```

Salt

The salt is combined with the secret key to derive a per-context signing key. It does not need to be secret or random — just **unique per purpose**. Without distinct salts, a token issued for "activate account" could be replayed as an "upgrade account" token if both sign the same data.

```
activate = URLSafeSerializer(KEY, salt="activate")
upgrade  = URLSafeSerializer(KEY, salt="upgrade")
upgrade.loads(activate.dumps(42))   # BadSignature
```

Defaults: `Serializer` uses `b"itsdangerous"`; `Signer` uses `b"itsdangerous.Signer"`. Passing `salt=None` to a Serializer falls through to the Signer default (which produces *different* tokens than the Serializer default).

Digest method

Default is HMAC with SHA-1. SHA-1's collision weaknesses don't apply inside HMAC, so this is safe. To use SHA-256/512 anyway, pass `signer_kwargs={"digest_method": hashlib.sha256}` (see §5 for migrating existing tokens). SHA-1 is looked up lazily so FIPS builds without SHA-1 can configure an alternative before first use.

---

3. Token formats

Knowing the wire format helps when debugging. All base64 is URL-safe with `=` padding stripped.

Class	Format
`Signer`	`value.SIG`
`TimestampSigner`	`value.TS.SIG` (SIG covers `value.TS`)
`Serializer`	`<json.dumps output>.SIG` (plain, not base64), e.g. `[1, 2, 3, 4].OiNT6D…`
`URLSafeSerializer`	`B64(compact-json).SIG`
`URLSafeSerializer` (compressed)	`.B64(zlib(json)).SIG` — leading `.` marks zlib
`URLSafeTimedSerializer`	`B64payload.TS.SIG`

- `TS` = base64 of the Unix timestamp as big-endian bytes (leading zero bytes stripped).
- URL-safe serializers use compact JSON (`separators=(",", ":")`, `ensure_ascii=False`) and only compress when it saves more than 1 byte.
- Separator defaults to `.`; it may not be a base64 character (letters, digits, `-_=`), or `Signer` raises `ValueError`.

Signing key derivation (`Signer.key_derivation`, default `"django-concat"`):

Value	Derived key
`django-concat` (default)	`digest(salt + b"signer" + secret_key)`
`concat`	`digest(salt + secret_key)`
`hmac`	`HMAC(secret_key, salt, digest)`
`none`	`secret_key` as-is (supported in code, not mentioned in docstrings)

The signature is then `HMAC(derived_key, value, digest_method)`, compared with `hmac.compare_digest` (constant time).

---

4. API reference

Everything below is importable from the top-level `itsdangerous` package.

`Signer(secret_key, salt=b"itsdangerous.Signer", sep=b".", key_derivation=None, digest_method=None, algorithm=None)`

Method	Returns	Notes
`sign(value)`	`bytes`	`value + sep + signature`
`unsign(signed_value)`	`bytes`	Raises `BadSignature` (with `.payload`)
`validate(signed_value)`	`bool`	Never raises
`get_signature(value)`	`bytes`	base64 signature only
`verify_signature(value, sig)`	`bool`	Tries every key in `secret_keys`
`derive_key(secret_key=None)`	`bytes`	Override to customise derivation

Class attributes to override in subclasses: `default_digest_method`, `default_key_derivation`. `algorithm` accepts any `SigningAlgorithm` (`HMACAlgorithm(digest)`, or `NoneAlgorithm` — empty signature, effectively no security; only for testing).

`TimestampSigner(...)` — same constructor as `Signer`

Method	Notes
`sign(value)`	Appends current Unix time
`unsign(value, max_age=None, return_timestamp=False)`	Returns `bytes` or `(bytes, datetime)`; datetime is timezone-aware UTC
`validate(value, max_age=None)`	`bool`
`get_timestamp()`	Override to control "now" (e.g. in tests)

`max_age=None` means **no expiry check**. A timestamp in the future (negative age) also raises `SignatureExpired`.

`Serializer(secret_key, salt=b"itsdangerous", serializer=None, serializer_kwargs=None, signer=None, signer_kwargs=None, fallback_signers=None)`

Method	Notes
`dumps(obj, salt=None)`	Returns `str` for text serializers (JSON), `bytes` for binary ones
`loads(s, salt=None)`	Raises `BadSignature` / `BadPayload`
`dump(obj, f, salt=None)` / `load(f, salt=None)`	File variants
`loads_unsafe(s, salt=None)`	`(signature_ok: bool, payload)`; never raises — **debug only**
`load_payload(payload)` / `dump_payload(obj)`	Serialization step only, no signing
`make_signer(salt=None)`	Builds the configured signer

- `serializer` — any object with `dumps`/`loads` (default stdlib `json`). Whether output is `str` or `bytes` is detected by calling `serializer.dumps({})`.
- `signer` / `signer_kwargs` — which Signer class to build and its arguments (`sep`, `digest_method`, `key_derivation`, …).
- Class attributes to override in subclasses: `default_serializer`, `default_signer`, `default_fallback_signers`.

`TimedSerializer(...)` — same constructor as `Serializer`

`loads(s, max_age=None, return_timestamp=False, salt=None)` and `loads_unsafe(s, max_age=None, salt=None)`. ⚠ Argument order differs from `Serializer.loads` — **pass `salt=` by keyword**.

`URLSafeSerializer`, `URLSafeTimedSerializer`

Same APIs as `Serializer` / `TimedSerializer`, but output is compact JSON → optional zlib → base64url. Output is always `str`. Safe for URLs, cookies, and HTML attributes.

Encoding helpers

`base64_encode(s) -> bytes`, `base64_decode(s) -> bytes` (raises `BadData`), `want_bytes(s, encoding="utf-8")`.

Exceptions

```
Exception
└── BadData                 .message          — base class for everything
    ├── BadPayload          .original_error   — signature OK (or skipped) but deserializing failed
    └── BadSignature        .payload          — signature mismatch / missing separator
        ├── BadHeader       .header, .original_error   (legacy; nothing in the package raises it now)
        └── BadTimeSignature .date_signed     — timed signature invalid or malformed timestamp
            └── SignatureExpired              — valid signature but older than max_age
```

Catch order matters: catch `SignatureExpired` before `BadSignature`, or just catch `BadData` to reject anything bad.

`e.payload` on `BadSignature` holds the **unverified** raw bytes. Only deserialize it (`s.load_payload(e.payload)`) for logging/debugging, and never with an unsafe serializer like `pickle`.

---

5. Common recipes

Email confirmation / password reset link

```
s = URLSafeTimedSerializer(app_secret, salt="email-confirm")
url = f"https://example.com/confirm/{s.dumps(user.email)}"
# later
email = s.loads(token, max_age=86400)
```

Find out when a token was signed

```
data, signed_at = s.loads(token, max_age=3600, return_timestamp=True)

try:
    s.loads(token, max_age=60)
except SignatureExpired as e:
    print("expired; signed at", e.date_signed)
```

Upgrade the digest without invalidating old tokens (fallback signers)

```
import hashlib
s = URLSafeSerializer(
    SECRET_KEY,
    signer_kwargs={"digest_method": hashlib.sha256},          # used to sign
```

```
    fallback_signers=[{"digest_method": hashlib.sha1}],         # also accepted when loading
)
```

Each `fallback_signers` item can be a `dict` of signer kwargs, a `Signer` subclass, or a `(SignerClass, kwargs)` tuple. Fallbacks only apply to `loads`; `dumps` always uses the primary signer. A `SignatureExpired` from the primary signer is raised immediately — fallbacks are not tried.

Rotate secret keys

```
keys = ["2023-key", "2024-key", "2025-key"]   # oldest -> newest
s = URLSafeSerializer(keys)                    # signs with "2025-key", accepts all three
```

Custom serializer — any object with `dumps`/`loads`:

```
s = Serializer(SECRET_KEY, serializer=my_msgpack_like, salt="x")   # bytes in, bytes out
```

Avoid `pickle`: if the key ever leaks, an attacker gets remote code execution.

Sign a plain string

```
signer = Signer(SECRET_KEY)
signer.sign("user-42")            # b'user-42.<sig>'
signer.unsign(b"user-42.<sig>")   # b'user-42'
```

---

6. Repository layout

```
src/itsdangerous/
  __init__.py      public re-exports (the whole public API)
  signer.py        SigningAlgorithm, HMACAlgorithm, NoneAlgorithm, Signer, key derivation
  timed.py         TimestampSigner, TimedSerializer
  serializer.py    Serializer (generic over str|bytes for typing), fallback signer logic
  url_safe.py      URLSafeSerializerMixin (zlib + base64), URLSafe[Timed]Serializer
  encoding.py      base64 helpers, int<->bytes for timestamps
  exc.py           exception hierarchy
  _json.py         compact JSON wrapper used by URL-safe serializers
  py.typed         PEP 561 marker — package ships type hints
tests/test_itsdangerous/   pytest suite, one file per module
docs/                      Sphinx sources (published on Read the Docs)
CHANGES.rst                changelog
pyproject.toml             metadata, tool config, tox envs
uv.lock                    locked dev dependencies
.devcontainer/             VS Code dev container (installs uv, runs uv sync + pre-commit)
.github/workflows/         tests, pre-commit, publish, lock
```

Class hierarchy:

```
Signer ──────────► TimestampSigner
Serializer ──────► TimedSerializer
URLSafeSerializerMixin + Serializer      = URLSafeSerializer
URLSafeSerializerMixin + TimedSerializer = URLSafeTimedSerializer
```

---

7. Development workflow

The project uses uv and tox (with `tox-uv`).

```
uv sync                          # create .venv with dev, pre-commit, tests, typing groups
uv run pytest                    # run tests
uv run tox run -e py3.13         # tests in a specific interpreter
uv run tox run -e typing         # mypy (strict) + pyright + pyright --verifytypes
uv run tox run -e style          # pre-commit: ruff lint + ruff format, uv-lock, whitespace checks
uv run tox run -e docs           # Sphinx build with warnings as errors
uv run tox run -e docs-auto      # live-reloading docs server
```

```
uv run tox                      # everything in env_list
pre-commit install              # run hooks on every commit
```

Notes: - Tests need `freezegun` (in the `tests` dependency group). Running bare `pytest` without `uv sync` fails to collect `test_timed.py` and `test_url_safe.py`. - pytest is configured with `filterwarnings = ["error"]` — any warning fails the test run. - CI (GitHub Actions) runs tests on Python 3.10–3.13, PyPy 3.11, Windows and macOS, plus the typing job. - Build backend is `flit_core`. The version lives in `pyproject.toml`; there is no `__version__` (removed in 2.3) — use `importlib.metadata.version("itsdangerous")`. - Lint rules: ruff `B, E, F, I, UP, W`, isort with one import per line. - Changes go under the "Unreleased" heading in `CHANGES.rst`.

---

8. Gotchas and known quirks

- **Payloads are readable.** Signing ≠ encryption.
- **Tokens don't expire unless you pass `max_age`** to `loads`. The token itself stores only the signing time.
- **Tokens aren't single-use.** A valid token can be replayed until it expires. For one-time actions (password reset), also check server-side state — e.g. include a value that changes after use (password hash fragment, `last_login`) in the payload and verify it.
- **Use different salts** for every purpose.
- **`TimedSerializer.loads` parameter order** is `(s, max_age, return_timestamp, salt)` — `loads(token, "my-salt")` passes the salt as `max_age` and crashes with `TypeError`. Use keywords.
- **Wrong salt on a timed serializer** raises `BadTimeSignature` (a `BadSignature` subclass), not plain `BadSignature`.
- **`Serializer` and `Signer` default salts differ.** Tokens from `Serializer(k)` and `Serializer(k, salt=None)` aren't interchangeable.
- **`loads_unsafe` / `e.payload`** return unverified data. Debug only.
- Docs drift in the repo's Sphinx sources: `docs/serializer.rst` shows `Serializer.dumps` returning `bytes` (it returns `str` with the default JSON serializer), and its fallback-signer example leaves out the required `secret_key` argument.
- `URLSafeSerializerMixin.load_payload` accepts a `serializer=` keyword but ignores it.
- `pyproject.toml` `[tool.coverage.run] source = ["jinja2", "tests"]` looks like a copy-paste from Jinja; coverage of `itsdangerous` won't be measured as configured.